

รายงานสัปดาห์ที่ 7

Web Scraping: จาก Basic ถึง Agentic (Advanced)

สรุปเนื้อหา บันทึกผลแล็บทั้งสองระดับ และตอบคำถามสำคัญประจำสัปดาห์

ไฟล์แนบโค้ด: week7_basic_scraper.py, week7_agentic_scraper.py, week7_agentic_config.json

1. ผลลัพธ์การเรียนรู้ (Learning Outcomes)

1.1 แล็บ Basic: Introduction to Web Scraping

- เข้าใจแนวคิดของ web scraping รวมถึงข้อพิจารณาด้านจริยธรรมและกฎหมาย
- ใช้ไลบรารี requests เพื่อดาว์โหลดเนื้อหาเว็บเพจ
- ใช้ไลบรารี BeautifulSoup4 (bs4) เพื่อแปลง (parse) และดึงข้อมูลเฉพาะจาก HTML
- เขียนสคริปต์ scraper พื้นฐานที่ดึงข้อมูลจากเว็บไซต์จริงได้
- ระบุและแก้ปัญหา (debug) ที่พบทั่วไปในการทำ web scraping

1.2 แล็บ Advanced: Agentic Web Scraper

- ใช้เทคนิค web scraping ขั้นสูงสำหรับเนื้อหาที่ต้องเรนเดอร์ด้วย JavaScript ผ่าน Selenium
- นำ error handling, retry mechanism และกลยุทธ์ anti-blocking พื้นฐานมาใช้
- ออกแบบและใช้ไฟล์ configuration เพื่อกำหนดพฤติกรรม scraper แบบ "agentic"
- จัดโครงสร้างข้อมูลที่ scrape มาและจัดเก็บเป็น JSON
- เข้าใจความรับผิดชอบด้านจริยธรรมและกฎหมายของการ scraping แบบอัตโนมัติขั้นสูง

2. สรุปเนื้อหาบทเรียน

2.1 Web Scraping คืออะไร และควรทำเมื่อไร

Web Scraping คือการดึงข้อมูลจากเว็บไซต์แบบอัตโนมัติด้วยโปรแกรม นำไปใช้ได้หลายด้าน เช่น การวิจัยตลาด การรวบรวมข่าว หรือการเปรียบเทียบราคา แต่ก่อนทำเสมอต้องตรวจสอบ robots.txt ของเว็บไซต์ (เช่น <https://example.com/robots.txt>) และ Terms of Service ว่าอนุญาตให้ scrape หรือไม่ ต้องเคารพ rate limiting (ไม่ยิง request รัว ๆ) และคำนึงถึงทรัพย์สินทางปัญญาของข้อมูล หากเว็บไซต์มี API ให้ใช้งาน ควรใช้ API แทนการ scrape เสมอ เพราะเสถียรกว่าและถูกต้องตามกฎหมายกว่า

2.2 พื้นฐาน HTTP สำหรับการ Scraping

- หลักการทำงานของเบราว์เซอร์คือวงจร request/response
- HTTP Method หลักที่ใช้คือ GET
- HTTP Status Code ที่พบบ่อย: 200 OK (สำเร็จ), 404 Not Found (ไม่พบหน้า), 403 Forbidden (ถูกปฏิเสธ), 500 Server Error (เซิร์ฟเวอร์ผิดพลาด)

2.3 ดาว์โหลดหน้าเว็บด้วย requests

- ติดตั้งด้วย `pip install requests`

- ดาวน์โหลดหน้าเว็บด้วย `requests.get(url)`
- เข้าถึงเนื้อหาที่ได้ผ่าน `.text` (ข้อความ) หรือ `.content` (ไบนารี)
- ใช้ `.raise_for_status()` เพื่อตรวจสอบว่า request สำเร็จหรือไม่ ถ้าไม่สำเร็จจะโยน error ออกมาให้ดักจับ
- ควรใส่ header `User-Agent` เลียนแบบเบราว์เซอร์จริง เพราะบางเว็บไซต์บล็อก request ที่ไม่มี header นี้

2.4 แปลงและดึงข้อมูลด้วย BeautifulSoup4

- ติดตั้งด้วย `pip install beautifulsoup4`
- สร้างอ็อบเจกต์แปลง HTML ด้วย `BeautifulSoup(html_doc, 'html.parser')`
- เข้าถึงแท็กด้วย `soup.tag_name`, เข้าถึง attribute ด้วย `tag['attribute']`, และดึงข้อความด้วย `tag.get_text()`
- `find()` หาอีลีเมนต์แรกที่ตรงเงื่อนไข ส่วน `find_all()` หาทุกอีลีเมนต์ที่ตรงเงื่อนไข โดยค้นได้ทั้งจากชื่อแท็ก, `id`, `class`
- `select()` ใช้ CSS selector ค้นหาข้อมูลได้ยืดหยุ่นกว่า เช่น `soup.select('div.product-name a')`

2.5 เนื้อหาขั้นสูง: Selenium สำหรับเว็บไดนามิก

`requests` และ `BeautifulSoup` ใช้ได้ดีกับ HTML แบบ static แต่ถ้าเนื้อหาถูกโหลดด้วย JavaScript, มี infinite scrolling หรือต้องคลิกปุ่มก่อนข้อมูลจะปรากฏ จำเป็นต้องใช้ `Selenium` ซึ่งเป็นเครื่องมือควบคุมเบราว์เซอร์จริงผ่านโค้ด (browser automation)

- ติดตั้งด้วย `pip install selenium` และตั้งค่า WebDriver ตามเบราว์เซอร์ (เช่น `ChromeDriver` ผ่าน `webdriver-manager`)
- ควบคุมเบราว์เซอร์พื้นฐาน: เปิด, นำทางด้วย `driver.get()`, ปิด
- ค้นหาอีลีเมนต์ด้วย `find_element(By.ID, ...)`, `By.CSS_SELECTOR`, หรือ `By.XPATH`
- โต้ตอบกับอีลีเมนต์: คลิกด้วย `.click()`, พิมพ์ข้อความด้วย `.send_keys()`, อ่านข้อความด้วย `.text`
- กลยุทธ์การรอ: `Implicit Wait` (timeout ทั่วไป) และ `Explicit Wait` ด้วย `WebDriverWait + expected_conditions` ซึ่งสำคัญมากสำหรับเนื้อหาไดนามิก
- Headless Browser คือการรันเบราว์เซอร์แบบไม่มีหน้าต่างให้เห็น (`--headless`) เพื่อความเร็วและประหยัดทรัพยากร

2.6 ความแข็งแรงของระบบและการป้องกันการถูกบล็อก

- หน่วงเวลาระหว่าง request ด้วย `time.sleep()` หรือใช้ explicit wait เพื่อความน่าเชื่อถือที่ดีกว่า
- ปลอม `User-Agent` ให้เหมือนเบราว์เซอร์จริง (User-Agent Spoofing)
- ทำ retry แบบพื้นฐานด้วย loop ร่วมกับ try-except เมื่อเจอข้อผิดพลาดชั่วคราว
- แนวคิดการหมุนเวียน IP (IP Rotation) ผ่าน proxy/VPN — เป็นแนวคิดขั้นสูงที่กล่าวถึงโดยสังเขปเท่านั้น

2.7 แนวคิด Agentic Scraping (Config-Driven Agent)

Agent คือระบบที่มีเป้าหมาย รับรู้สภาพแวดล้อม ดำเนินการ และปรับตัวได้ ในบริบทนี้ เราจำลองพฤติกรรมแบบ agent โดยกำหนด "กฎ" การ scrape ไว้ในไฟล์ configuration (.json) แทนการฝังค่าตายตัวในโค้ด เช่น start_url, pagination_selector (selector ของปุ่มหน้าถัดไป), item_container_selector (selector ของบล็อกสินค้าหนึ่งชิ้น) และ item_data_selectors (selector ของแต่ละฟิลด์ข้อมูล) จากนั้น agent จะอ่าน config แล้ววนลูป: นำทาง → ดึงข้อมูล → หาหน้าถัดไป → ทำซ้ำ จนไม่มีหน้าเหลือหรือถึงขีดจำกัดที่กำหนด

3. สรุปผลการทำแล็บ

3.1 Basic Lab: Simple Web Scraper

โค้ดอยู่ในไฟล์แนบ week7_basic_scraper.py เขียนฟังก์ชัน get_html_content() ดาวน์โหลด HTML ด้วย requests พร้อมดักจับข้อผิดพลาด (เช่น เชื่อมต่อไม่ได้, timeout, status code ผิดพลาด) และฟังก์ชัน scrape_book_and_chapters() ใช้ BeautifulSoup ดึงชื่อหนังสือจากแท็ก <h1> ภายใน <article>><header> และดึงรายชื่อบททั้งหมดจาก <div class="content-body"> เป้าหมายคือหน้าแรกของ "Automate the Boring Stuff with Python" (automatetheboringstuff.com/3e/)

ทดสอบรันจริงแล้ว โปรแกรมดาวน์โหลดสำเร็จ (status code 200) และดึงชื่อหนังสือ "3rd Edition" พร้อมชื่อบททั้งหมด 27 รายการ ได้ถูกต้องตรงกับโครงสร้างหน้าเว็บจริง

3.2 Advanced Lab: Agentic Product Scraper

โค้ดอยู่ในไฟล์แนบ week7_agentic_scraper.py ทำงานร่วมกับไฟล์ config week7_agentic_config.json โดยฟังก์ชัน create_browser() เปิด Chrome ผ่าน Selenium แบบ headless พร้อมติดตั้ง ChromeDriver อัตโนมัติด้วย webdriver-manager ฟังก์ชัน run_agent() เป็นหัวใจของ agent: นำทางไปยัง start_url ในไฟล์ config, วนลูปดึงข้อมูลสินค้าแต่ละหน้าตาม item_container_selector และ item_data_selectors, แล้วคลิกปุ่มหน้าถัดไปตาม pagination_selector จนครบ max_pages หรือหาไม่เจอ ผลลัพธ์ทั้งหมดถูกบันทึกลงไฟล์ scraped_products.json

ไฟล์นี้ทำหน้าที่เหมือนไฟล์ scraper_agent.py + driver_manager.py + config_parser.py

ในเอกสารประกอบการสอน แต่รวมเป็นฟังก์ชันง่าย ๆ ไว้ในไฟล์เดียว

เพื่อให้อ่านและเข้าใจง่ายขึ้นสำหรับผู้เริ่มต้น config ตัวอย่างที่แนบมาชี้ไปที่ books.toscrape.com ซึ่งเป็นเว็บไซต์ที่สร้างขึ้นมาเพื่อฝึกฝนการ scrape โดยเฉพาะ การรันไฟล์นี้ต้องติดตั้ง pip install selenium webdriver-manager และมี Google Chrome อยู่ในเครื่องก่อน

4. คำถาม-คำตอบสำคัญประจำสัปดาห์

4.1 ทำไมต้องตรวจสอบ robots.txt และ Terms of Service ก่อน scrape ทุกครั้ง?

เพราะ robots.txt เป็นไฟล์ที่เจ้าของเว็บไซต์ใช้ระบุว่าจะอนุญาตให้ bot เข้าถึงส่วนใดได้บ้าง ส่วน Terms of Service คือข้อตกลงการใช้งานที่มีผลทางกฎหมาย หากเว็บไซต์ห้าม scraping ไว้ชัดเจนแล้วยังฝ่าฝืน อาจถือเป็นการละเมิดสัญญา และนำไปสู่การถูกบล็อก IP หรือถูกดำเนินคดีได้ การตรวจสอบก่อนเสมอจึงเป็นทั้งเรื่องจริยธรรมและการป้องกันความเสี่ยงทางกฎหมาย

4.2 ควรใช้ requests + BeautifulSoup หรือ Selenium เมื่อไร?

ถ้าข้อมูลที่ต้องการอยู่ใน HTML ตั้งแต่แรกที่โหลดหน้า (static content) ให้ใช้ requests + BeautifulSoup เพราะเร็วกว่า ใช้ทรัพยากรน้อยกว่า และเขียนโค้ดง่ายกว่ามาก แต่ถ้าข้อมูลถูกสร้างขึ้นภายหลังด้วย JavaScript เช่น ต้องเลื่อนหน้าจอ (infinite scroll) หรือต้องคลิกปุ่มก่อนข้อมูลจะปรากฏ (dynamic content)

กรณีนี้ requests จะดึงมาได้แค่ HTML เปล่า ๆ จำเป็นต้องใช้ Selenium ซึ่งควบคุมเบราว์เซอร์จริงที่รันJavaScript ได้ครบถ้วน

4.3 การแก้ปัญหาที่พบบ่อยมีอะไรบ้าง?

- `AttributeError: 'NoneType' object has no attribute ...` — เกิดจาก `find()/select()` หา element ไม่เจอแล้วคืนค่า `None` วิธีแก้คือตรวจสอบด้วย `if element`: ก่อนเข้าถึงข้อมูลข้างในเสมอ
- `403 Forbidden` — เว็บไซต์บล็อก request ที่ไม่มี User-Agent วิธีแก้คือเพิ่ม header User-Agentปลอมเป็นเบราว์เซอร์จริง
- `ModuleNotFoundError` — ยังไม่ได้ activate virtual environment หรือยังไม่ได้ติดตั้งไลบรารี วิธีแก้คือ activate venv แล้ว `pip install` ให้ครบ
- ถูกบล็อกหลัง request ถูเกินไป — วิธีแก้คือเพิ่ม `time.sleep()` หน่วงเวลาระหว่างแต่ละ request

4.4 แล็บ Basic กับ Advanced (Agentic) ต่างกันอย่างไร?

แล็บ Basic เหมาะกับเว็บไซต์ static ที่ข้อมูลอยู่ครบใน HTML ตั้งแต่แรก ใช้แค่ requests + BeautifulSoup ก็เพียงพอ และโค้ดมีโครงสร้างตายตัว (hard-coded) เจาะจงกับเว็บไซต์เดียว ส่วนแล็บ Advanced ยกกระดับขึ้นสองด้าน: (1) ใช้ Selenium เพื่อรองรับเว็บไซต์ที่มีเนื้อหาไดนามิกและมีหลายหน้า (pagination) และ (2) เปลี่ยนจากการ hard-code มาเป็นการอ่านกฎการ scrape จากไฟล์ config ภายนอก ทำให้นาสcraper ตัวเดียวกันไปใช้กับเว็บไซต์อื่นได้ทันทีเพียงแค่แก้ไฟล์ config โดยไม่ต้องแก้โค้ด ซึ่งเป็นจุดเริ่มต้นของแนวคิดการออกแบบระบบอัตโนมัติแบบ agent

5. บทสรุป

สัปดาห์นี้ต่อยอดจากการเขียนโปรแกรมพื้นฐานไปสู่การดึงข้อมูลจากโลกภายนอกจริง ๆ เริ่มจาก scraping แบบง่ายด้วย requests และ BeautifulSoup สำหรับเว็บ static ไปจนถึงการใช้ Selenium ควบคุมเบราว์เซอร์สำหรับเว็บไดนามิก และปิดท้ายด้วยแนวคิด config-driven agent ที่ทำให้ scraper ยืดหยุ่นและนำกลับมาใช้ซ้ำได้กับหลายเว็บไซต์ ตลอดกระบวนการนี้สิ่งที่สำคัญไม่แพ้เทคนิค คือความรับผิดชอบด้านจริยธรรมและกฎหมาย ทั้งการเคารพ robots.txt, Terms of Service และการ scrape อย่างสุภาพไม่เป็นการต่อเซิร์ฟเวอร์ปลายทาง

อ้างอิง: เอกสารประกอบการสอน "LabsX - Basic: Introduction to Web Scraping" และ "Agentic Web Scraper ด้วย Python และ Selenium"